

基础编译器 常用编译选项手册

V1.4.2

2020 年 6 月 22 日

目录

第一章	概述.....	5
第二章	功能选项.....	6
1.	-mhost	6
2.	-mslave	6
3.	-mhybrid.....	6
4.	-msimd	6
5.	-mhybrid_dynamic	6
6.	-faddress_align= <i>n</i>	7
7.	-c	7
8.	-S	7
9.	-E.....	8
10.	-o <i>file</i>	8
11.	-fcheck-ldm-stack	8
12.	-fstack-protector	8
13.	-fstack-arrays.....	8
14.	-finit-local-zero	8
15.	-ffree-form.....	9
16.	-fconvert='big-endian'	9
17.	-mieee.....	9
18.	-mieee-with-inexact	10
19.	-mftz.....	10
20.	-masm-volatile	10
21.	-fgnu89-inline	10
22.	-fno-tree-ch	10
23.	-fstrict-aliasing	10
24.	-fno-common.....	11
25.	-funconstrained-commons.....	11
26.	-fslave-default-thread	11

27.	-fdec-math	11
28.	-Werror=xxx	11
29.	--params bbss_data_min_size=xxx	13
30.	-fextract-feature	13
31.	-ffree-line-length-<n>	15
第三章 性能选项.....		16
1.	-O[0/1/2/3/s].....	16
2.	-Ofast.....	20
3.	-ffast-math.....	21
4.	-ffp-contract=style	21
5.	-funroll-loops --param max-unroll-times= <i>n</i>	21
6.	-funroll-all-loops	21
7.	-fprefetch-loop-arrays	22
8.	-flto.....	22
9.	-mslave-lto	22
10.	-fwhole-program	22
11.	-fipa-type-escape -fipa-struct-peeling	22
12.	-Wl,--gc-sections.....	23
13.	-fbranch-predict-hint	23
14.	-mpws.....	23
15.	-fmodule-ldm	23
16.	-mfma.....	23
17.	-msimd -ftree-vectorize	24
18.	-fcache-hint.....	24
19.	-fdynamic-uncache-symbols	24
20.	-lm、-lm_slave.....	26
第四章 调试选项.....		27
1.	-fverbose-asm.....	27
2.	-dp/-dP.....	27
3.	-g	27

4.	-pg	27
5.	-fswperf	27
6.	-fdump-tree-all	27
7.	-fdump-rtl-all.....	28
8.	-fdump-ipa-all	28
第五章 其他选项.....		29
1.	为特定函数设置编译优化选项.....	29
2.	自动向量化相关选项.....	30
3.	功耗相关优化.....	31

第一章 概述

本文描述 SWGCC 编译器支持的常用编译选项。

按照章节顺序分为功能选项、性能相选项、调试选项以及其他选项。

更全的编译选项可通过 `man gcc` 命令查看。

第二章 功能选项

1. -mhost

描述:

指示编译器生出 SW39000 众核处理器主核代码。

备注:

- 1) 打开预定义宏 `_SIMD_HOST`;
- 2) `swgcc` 缺省使用 `-mhost` 生成主核代码;
- 3) 对应 `swcc` 编译器的 `-host` 选项。

2. -mslave

描述:

指示编译器生出 SW39000 众核处理器从核代码。

备注:

- 1) 打开预定义宏 `_SIMD_SLAVE`;
- 2) 禁止同时打开 `-mhost` 和 `-mslave` 选项，否则编译器报错退出;
- 3) 对应 `swcc` 编译器 `-slave` 选项。

3. -mhybrid

描述:

指示链接器进行主从核代码混合链接，链接对象是目标代码.o 文件。

备注:

对应 `swcc` 编译器 `-hybrid` 选项。

示例:

```
swgcc -mhybrid master.o slave.o -o ./a.out
```

4. -msimd

描述:

指示编译器支持 SIMD 扩展编程接口，生成相应的 SIMD 扩展指令。

备注:

对应 `swcc` 编译器 `-simd` 选项，不再包含地址对界功能，对界功能由下面的 `-faddress_align=n` 选项完成。

5. -mhybrid_dynamic

描述:

编译器支持代码的动态链接。主核默认采用动态链接模式，从核默认采用静态链接模式。使用 `-mhybrid_dynamic` 可支持主从混合的动态链接模式。

备注:

使用动态库的程序通过 `dlopen_hybrid` 打开动态库。

示例:

1、生成动态库

```
swgcc -mhost -c host.c
```

```
swgcc -mslave -fPIC -shared -c slave.c
```

```
swgcc -mhybrid_dynamic host.o slave.o -shared -o test_hybrid.so
```

2、使用动态库

```
swgcc -c dl.c
```

```
swgcc -mhybrid_dynamic dl.o -ldl -o dl_test
```

```
//示例程序 dl.c
```

```
#include <stdio.h>
```

```
#include <stdlib.h>
```

```
#include <dlfcn.h>
```

```
void* dlopen_hybrid(char*,int);
```

```
int main(int argc, char **argv){
```

```
    void *handle;
```

```
    int (*test_hybrid)();
```

```
    handle = dlopen_hybrid("./test_hybrid.so", RTLD_NOW);
```

```
    dLError();    /* Clear any existing error */
```

```
    printf("dlopen ./test_hybrid.so succeeded!\n\n");
```

```
    *(void **)(&test_hybrid) = dlsym(handle, "test_hybrid");
```

```
    printf("call test_hybrid function in test_hybrid.so!\n\n");
```

```
    (*test_hybrid)();
```

```
    dlclose(handle);
```

```
    exit(EXIT_SUCCESS);
```

```
}
```

6. -faddress_align=*n*

描述:

指示编译器将所有数组、向量、结构体、联合体等的首地址按照 *n* B 对界。

备注:

该选项剥离出原来的 -msimd 选项的对界功能, 一般情况下, 主核程序设置 *n* 为 32, 从核程序设置 *n* 为 64。

7. -c

描述:

指示编译器只对源文件进行编译和汇编, 生成目标文件, 不进行链接。

备注:

- 1) 源文件格式为 '.c', '.i', '.s' 等, 目标文件格式为 '.o';
- 2) 对应 swcc 编译器 -c 选项。

8. -S

描述:

指示编译器只对源文件进行编译, 生成汇编文件, 不进行汇编和链接。

备注:

- 1) 源文件格式为 '.c', '.i' 等, 汇编文件格式为 '.s';

- 2) 对应 swcc 编译器-S 选项。

9. -E

描述:

指示编译器仅对源文件进行预处理, 不进行编译、汇编和链接。

备注:

- 1) 源文件格式为'.c'等, 预处理后的源文件格式为'.i';
- 2) 对应 swcc 编译器-cpp 或 swf90 编译器-ftpp 选项。

10. -o *file*

描述:

指示编译器将输出内容存放在命名为 *file* 的文件中。

备注:

- 1) 该选项在生成预处理的源文件、汇编文件、目标文件及可执行文件的过程均可使用;
- 2) 对应 swcc 编译器-o 选项。

11. -fcheck-ldm-stack

描述:

当从核栈空间放在 LDM 上时, 运行过程中每个从核函数一开始先检测栈空间是否会溢出到用户 LDM 空间, 如果溢出, 则进入报错函数, 否则正常执行。

12. -fstack-protector

描述:

启用堆栈保护功能, 但是只为局部变量包含 char 类型数组的函数或者含有 alloca 调用的函数打开。

备注:

该选项包含以下变体:

- 1) -fstack-protector-all: 为所有函数打开堆栈保护功能;
- 2) -fstack-protector-strong: 使用更加聪明的策略为某些函数打开堆栈保护功能;
- 3) -fstack-protector-explicit: 仅为含有 “stack_protect” attribute 的函数打开堆栈保护功能, 如: void test() __attribute__((stack_protect))。

13. -fstack-arrays

描述:

指示编译器将所有局部数组存放到栈空间, 即使局部数组的大小未知。

备注:

- 1) 该选项为 swgfortran 选项;
- 2) 当打开 ‘-Ofast’ 优化选项时, 该选项默认打开。

14. -finit-local-zero

描述:

指示编译器将 INTEGER、REAL 以及 COMPLEX 变量置零, 逻辑变量置为 false, CHARACTER 变量置为一个空字符串。

备注:

- 1) 该选项为 swgfortran 选项;
- 2) 对应的 swf90 编译器-zerouv 选项。

示例:

swgfortran -S -O0 -finit-local-zero a.f90 -o 1.s 得到 1.s。

15. -ffree-form

描述:

强制对源文件按照自由格式进行语法分析, 默认.f90、.F90、.f95、.F95 文件被认为自由格式。

备注:

- 1) 该选项为 swgfortran 选项
- 2) 对应 swf90 编译器-freeform 选项

示例:

创建一个 b.f 文件, 该源文件不是自由格式文件, 因此正常情况下不加-ffreeform 选项无法对其进行语法分析。

swgfortran -O0 b.f 得到结果报 error;

swgfortran -O0 -ffree-form b.f 能够编译完成, 得到可执行文件, 且保证正确性。

16. -fconvert='big-endian'

描述:

输入输出设置为大端格式。

备注:

- 1) 该选项为 swgfortran 选项;
- 2) 对应 swf90 编译器-convert big_endian 选项;
- 3) 该选项的形式为-fconvert=conversion, conversion 可以为 native, swap, big-endian, little-endian;
- 4) -fconvert='big-endian'这个选项是编译选项, 相当于运行时的 GFORTRAN_CONVERT_UNIT='big_endian' ./a.out。

示例:

swgfortran -fconvert='big-endian' -S -O0 b.f90 -o 1.s 得到文件 1.s。

17. -mieee

描述:

指示编译器允许使用产生精确结果的任何操作。对于非精确结果的情况会影响精度。例如, $X*0$ 可以换成 0, X/X 可以换成 1, 虽然当 X 为+inf, -inf 或 NaN 时结果是不准确的。

备注:

- 1) 该选项仅适用于主核程序, 从核硬件支持非规格化数, 不需要使用该选项;
- 2) 对应 swf90 编译器-OPT:IEEE_arith=2 选项。

示例:

swgfortran -S -O0 -mieee c.f90 -o 1.s

swgcc -S -O0 -mieee a.c -o 1.s

18. -mieee-with-inexact

描述:

指示编译器严格遵从 IEEE 754 标准。

备注:

- 1) 该选项仅适用于主核程序，从核硬件支持非规格化数，不需要使用该选项；
- 2) 对应 swf90 编译器-OPT:IEEE_arith=1 选项。

19. -mftz

描述:

指示编译器在发生下溢时，将真零（64 位全“0”）写入目标寄存器，同时将非规格化的源操作数当做有符号的“0”处理。

备注:

- 1) 该选项仅适用于主核程序，从核硬件支持非规格化数，不需要使用该选项；
- 2) 该选项仅在打开-mieee 或-mieee-with-inexact 选项时有效；
- 3) 对应 swcc 编译器-ftz 选项。

20. -masm-volatile

描述:

指示编译器将程序中所有的嵌汇编设置 volatile 属性，不允许对其进行优化。

备注:

经验不足的程序员在使用嵌汇编时出现低级错误，查错难度大，如果程序中有嵌汇编且没有加上 volatile 属性，运行出错时可以先加上该选项排除错误。

21. -fgnu89-inline

描述:

指示编译器识别旧标准内联函数关键字“inline”。

备注:

内联函数 inline 是旧标准，新标准是__inline__。旧标准可以加-fgnu89-inline 识别。gcc 会内部识别函数是否会内联，如果要强制内联需要加 __attribute__((__always_inline__))。

22. -fno-tree-ch

描述:

该选项指示编译器关闭循环头部拷贝优化。循环头部拷贝优化在-O1 选项及以上打开。

备注:

用户在循环体内部写嵌汇编，嵌汇编里使用标号，若打开循环头部拷贝优化，则 pass_tree_ch 优化遍会把这个循环体的头部提到循环体外预先执行，导致嵌汇编代码也被复制，报标号重复定义。优化遍本身没有问题，主要是编译器对嵌汇编代码不做任何处理。使用-fno-tree-ch 选项关闭该优化可以解决。

23. -fstrict-aliasing

描述:

该选项指示编译器使用更严格的别名检查。该选项在-O3 以上优化下打开。

备注：

某些课题中存在不同类型的指针和变量的别名关系，更严格的别名检查机制会认为这些指针和变量不是别名，编译器认为没有关系，值并没有被修改，所以被优化掉，导致 O3 结果错误。使用-fno-strict-aliasing 选项关闭该优化可解决。

24. -fno-common

描述：

该编译指示将 comm 或者 bss 的未初始化变量强制初始化 0，这样程序中的多重符号定义将直接报错退出。

25. -funconstrained-commons

描述：

指示编译器在 common 块中声明的变量未来可能会被更长的数据覆盖，阻止编译器对超出 common 块边界的赋值语句进行优化。

备注：

该选项为 swgfortran 选项。

26. -fslave-default-thread

描述：

该选项指示编译器将从核全局变量默认添加__thread 属性，这样 busb 提交多道纯从核课题的全局变量就不会分配到同一段空间。

备注：

CPU 测试组提出需求。

27. -fdec-math

描述：

指示编译器兼容旧标准的三角函数接口，例如 sind 和 cosd 等。

备注：

该选项为 gcc710 新增选项，仅适用于 swgfortran 编译器。

28. -Werror=xxx

描述：

指示编译器将 xxx 类型的警告信息当做错误信息报出，提示用户修改程序。

备注：

1) 支持的警告类型包括：

-Waddress

-Warray-bounds=1 (only with '-O2')

-Wbool-compare

-Wbool-operation

-Wc++11-compat -Wc++14-compat

-Wchar-subscripts

-Wcomment

-Wduplicate-decl-specifier (C and Objective-C only)

- Wenum-compare (in C/ObjC; this is on by default in C++)
 - Wformat
 - Wint-in-bool-context
 - Wimplicit (C and Objective-C only)
 - Wimplicit-int (C and Objective-C only)
 - Wimplicit-function-declaration (C and Objective-C only)
 - Winit-self (only for C++)
 - Wlogical-not-parentheses
 - Wmain (only for C/ObjC and unless ‘-ffreestanding’)
 - Wmaybe-uninitialized
 - Wmemset-elt-size
 - Wmemset-transposed-args
 - Wmisleading-indentation (only for C/C++)
 - Wmissing-braces (only for C/ObjC)
 - Wnarrowing (only for C++)
 - Wnonnull
 - Wnonnull-compare
 - Wopenmp-simd
 - Wparentheses
 - Wpointer-sign
 - Wreorder
 - Wreturn-type
 - Wsequence-point
 - Wsign-compare (only in C++)
 - Wsizeof-pointer-memaccess
 - Wstrict-aliasing
 - Wstrict-overflow=1
 - Wswitch
 - Wtautological-compare
 - Wtrigraphs
 - Wuninitialized
 - Wunknown-pragmas
 - Wunused-function
 - Wunused-label
 - Wunused-value
 - Wunused-variable
 - Wvolatile-register-var
- 2) 建议用户程序加上-Werror=implicit-function-declaration，以防止出现返回值类型匹配错误的问题；
 - 3) 建议用户程序在-O3 情况下或者-fstrict-aliasing 使能情况下，开启-Wstrict-aliasing，以防止代码中违反规则的代码被错误地优化掉，从而导致程序结果出错。

29. --params bbss_data_min_size=xxx

描述:

设置放入 bbss 段内数据大小的最小阈值, 默认为 1024G。

备注:

由于 bss 段内数据过大会导致 gp 寻址无法访问部分数据, 新设置 bbss 段用于存在部分体积较大的数据, 数据大小的值通过编译选项配置。

30. --fextract-feature

描述:

基于控制流图的代码静态特征提取, 对代码的静态分析提供支持。

备注:

- 1) 以函数为单位进行静态特征收集;
- 2) 特征参数含义如下表所示, 共 65 项静态特征。

名称	类型	描述
ft1	CFG(basic blocks)	Number of basic blocks in the method
ft2	CFG(basic blocks)	Number of basic blocks with a single successor
ft3	CFG(basic blocks)	Number of basic blocks with two successors
ft4	CFG(basic blocks)	Number of basic blocks with more then two successors
ft5	CFG(basic blocks)	Number of basic blocks with a single predecessor
ft6	CFG(basic blocks)	Number of basic blocks with two predecessors
ft7	CFG(basic blocks)	Number of basic blocks with more then two predecessors
ft8	CFG(basic blocks)	Number of basic blocks with a single predecessor and a single successor
ft9	CFG(basic blocks)	Number of basic blocks with a single predecessor and two successors
ft10	CFG(basic blocks)	Number of basic blocks with a two predecessors and one successor
ft11	CFG(basic blocks)	Number of basic blocks with two successors and two predecessors
ft12	CFG(basic blocks)	Number of basic blocks with more then two successors and more then two predecessors
ft13	CFG(basic blocks)	Number of basic blocks with number of instructions in the interval [M, N] ($N < 15$, $[M, N] = [15, 500]$, $M > 500$)
ft14	CFG(basic blocks)	Number of basic blocks with number of instructions in the interval [M, N] ($N < 15$, $[M, N] = [15, 500]$, $M > 500$)
ft15	CFG(basic blocks)	Number of basic blocks with number of instructions in the interval [M, N] ($N < 15$, $[M, N] = [15, 500]$, $M > 500$)
ft16	CFG(edges)	Number of edges in the control flow graph
ft17	CFG(edges)	Number of critical edges
ft18	CFG(edges)	Number of abnormal edges
ft24	Instructions	Number of instructions in (method, loops)
ft25	Instructions	Average of number of instructions in basic blocks
ft19	Instructions	Number of direct calls in (method, loops)
ft39	Instructions	Number of indirect calls (i.e. done via pointers) in (method, loops)
ft20	Instructions	Number of conditional branches in (method, loops)
ft56	Instructions	Number of unconditional branches in (method, loops)

ft33	Instructions	Number of switch instructions in (method, loops)
ft21	Instructions	Number of assignments in (method, loops)
ft35	Instructions	Number of instructions that do pointer arithmetic in (method, loops)
ft22	Instructions	Number of binary integer operations in (method, loops)
ft23	Instructions	Number of binary floating point operations in (method, loops)
ft34	Instructions	Number of unary operations in (method, loops)
ft36	Instructions	Number of indirect references via pointers ('*' in C)
ft37	Instructions	Number of times the address of a variables is taken ('&' in C)
ft38	Instructions	Number of times the address of a function is taken ('&' in C)
ft40	Instructions	Number of assignment instructions with the right operand an integer constant in (method, loops)
ft41	Instructions	Number of binary operations with one of the operands an integer constant in (method, loops)
ft42	Instructions	Number of calls where the number of arguments of the callee is in the interval [M, N] ([M,N]=[0, 3], N > 4)
ft43	Instructions	Number of calls with pointers as arguments
ft44	Instructions	Number of calls that return a pointer
ft45	Instructions	Number of calls that return an integer
ft46	Constants	Number of occurrences of integer constant zero
ft48	Constants	Number of occurrences of integer constant one
ft47	Constants	Number of occurrences of 32-bit integer constants
ft49	Constants	Number of occurrences of 64-bit integer constants
ft51	Variables	Number of references (def/use) of a static/extern variables in (method, loops)
ft50	Variables	Number of references of a local variables in (method, loops)
ft52	Variables	Number of local variables referred in (method, loops)
ft53	Variables	Number of static/extern variables referred in (method, loops)
ft54	Variables	Number of local variables that are pointers in (method, loops)
ft55	Variables	Number of static/extern variables that are pointers in (method, loops)
ft26	PHI-NODES	Average of number of phi-nodes at the beginning of a basic block
ft27	PHI-NODES	Average of arguments for a phi-node
ft28	PHI-NODES	Number of basic blocks with the number of phi-nodes in the interval [M, N] ($M < 1$, $[M,N]=[1,3]$, $N>3$)
ft29	PHI-NODES	Number of basic blocks with the number of phi-nodes in the interval [M, N] ($M < 1$, $[M,N]=[1,3]$, $N>3$)
ft30	PHI-NODES	Number of basic blocks with the number of phi-nodes in the interval [M, N] ($M < 1$, $[M,N]=[1,3]$, $N>3$)
ft31	PHI-NODES	Number of basic block where total number of arguments for all phi-nodes is in the interval [M,N] ($[M,N]=[1, 5]$, $N > 5$)
ft32	PHI-NODES	Number of basic block where total number of arguments for all phi-nodes is in the interval [M,N] ($[M,N]=[1, 5]$, $N > 5$)
ft57	Loops	CYCLOMATIC COMPLEXITY
ft58	Loops	HALSTEAD's METRICS - HN2 is total number of operands
ft59	Loops	HALSTEAD's METRICS - Hn2 is number of distinct operands (Halstead n2)

ft60	Loops	HALSTEAD's METRICS - N is num var defs (should be == Halstead n2 or Halstead N2?)
ft61	Loops	HALSTEAD's METRICS - HN1 is total number of operators (Halstead N1) (approx due to abstraction)
ft62	Loops	HALSTEAD's METRICS - Hn1 is number of distinct operators (Halstead n1) (approx due to abstraction)
ft63	Loops	HALSTEAD's METRICS - approx of Halstead difficulty $D = Hn1/2 * (HN2 / Hn2)$ (NB uses the aver routine to avoid problems with 0 divisor in computation of $HN2/Hn2$)
ft64	Loops	HALSTEAD's METRICS - approx of Halstead volume (volume == $HN * \log_2(Hn)$ where $HN = HN1+HN2$ and $Hn = Hn1+Hn2$)
ft65	Loops	HALSTEAD's METRICS - approx of Halstead effort, which == Difficulty * Volume

31.-ffree-line-length-<n>

描述:

Fortran 语句允许每行最长 132 个字符, 如果超过 132 个字符, 可使用-ffree-line-length-<n>选项指定最长为 n 个字符, 如-ffree-line-length-300 指定每行最长为 300 个字符。

例如下列程序:

```

program test
integer ::status
call handle_error(status, __FILE__, __LINE__)
end program

```

如果由于目录过深或者文件名过长导致程序第三行超过 132 个字符, 这时可使用-ffree-line-length-<n>选项。

第三章 性能选项

1. -O[0/1/2/3/s]

a) -O0

描述:

该选项提供基础级别的优化，减少编译时间，生成调试所需信息。

备注:

1) 该选项使能下列优化选项:

- faddress_align
- faggressive-loop-optimizations
- fauto-inc-dec
- fchkp-check-incomplete-type
- fchkp-check-read
- fchkp-check-write
- fchkp-instrument-calls
- fchkp-narrow-bounds
- fchkp-optimize
- fchkp-store-bounds
- fchkp-use-static-bounds
- fchkp-use-static-const-bounds
- fchkp-use-wrappers
- fcommon
- fdata-sections
- fdelete-null-pointer-checks
- fdwarf2-cfi-asm
- fearly-inlining
- feliminate-unused-debug-types
- ffp-int-builtin-inexact
- ffunction-cse
- ffunction-sections
- fgcse-lm
- fgnu-runtime
- fgnu-unique
- fident
- finline-atomics
- fira-hoist-pressure
- fira-share-save-slots
- fira-share-spill-slots
- fivopts
- fkeep-static-consts
- fleading-underscore
- flifetime-dse
- flto-odr-type-merging

-fmath-errno
-fmerge-debug-strings
-fpcc-struct-return
-fpeephole
-fplt
-fprefetch-loop-arrays
-fsched-critical-path-heuristic
-fsched-dep-count-heuristic
-fsched-group-heuristic
-fsched-interblock
-fsched-last-insn-heuristic
-fsched-rank-heuristic
-fsched-spec
-fsched-spec-insn-heuristic
-fsched-stalled-insns-dep
-fschedule-fusion
-fsemantic-interposition
-fshow-column
-fshrink-wrap-separate
-fsigned-zeros
-fsplit-ivs-in-unroller
-fssa-backprop
-fstdarg-opt
-fstrict-volatile-bitfields
-fsync-libcalls
-ftrapping-math
-ftree-cselim
-ftree-forwprop
-ftree-loop-if-convert
-ftree-loop-im
-ftree-loop-ivcanon
-ftree-loop-optimize
-ftree-parallelize-loops=
-ftree-phirop
-ftree-reassoc
-ftree-scev-cprop
-funit-at-a-time
-funroll-innermost-loop
-funwind-tables

2) 对应 swcc 编译器-O0 选项。

b) -O1

描述:

该选项指示编译器会在不引入过多编译时间开销的前提下，生成更快更小的代码。

备注:

1) 该选项在-O0 基础上使能下列优化选项:

- fbranch-count-reg
- fcombine-stack-adjustments
- fcompare-elim
- fcprop-registers
- fdefer-pop
- fforward-propagate
- fguess-branch-probability
- fif-conversion
- fif-conversion2
- finline
- finline-functions-called-once
- fipa-profile
- fipa-pure-const
- fipa-reference
- fmerge-constants
- fmove-loop-invariants
- fomit-frame-pointer
- freorder-blocks
- fshrink-wrap
- fsplit-wide-types
- fssa-phiopt
- ftoplevel-reorder
- ftree-bit-ccp
- ftree-builtin-call-dce
- ftree-ccp
- ftree-ch
- ftree-coalesce-vars
- ftree-copy-prop
- ftree-dce
- ftree-dominator-opts
- ftree-dse
- ftree-fre
- ftree-pta
- ftree-sink
- ftree-slsr
- ftree-sra
- ftree-ter

2) 对应 swcc 编译器-O1 选项。

c) -O2

描述:

在-O1 的基础上的进一步优化。这是 SWGCC 的默认优化等级。-O2 会比-O1 多使能一些优化选项, 设置了-O2 后, 编译器试图提高代码性能而不会增大体积和大量占用编译时间。

备注:

1) 在-O1 基础上, -O2 使能了以下编译选项:

- falign-labels
- fcaller-saves
- fcode-hoisting
- fcrossjumping
- fcse-follow-jumps
- fdevirtualize
- fdevirtualize-speculatively
- fexpensive-optimizations
- fgcse-lm
- fhoist-adjacent-loads
- findirect-inlining
- finline-small-functions
- fipa-bit-cp
- fipa-cp
- fipa-icf
- fipa-icf-functions
- fipa-icf-variables
- fipa-ra
- fipa-sra
- fipa-vrp
- fisolate-erroneous-paths-dereference
- flra-remat
- foptimize-sibling-calls
- foptimize-strlen
- fpartial-inlining
- fpeepphole2
- free
- freorder-functions
- frerun-cse-after-loop
- fschedule-insns
- fschedule-insns2
- fstore-merging
- fstrict-overflow
- fthread-jumps
- ftree-pre
- ftree-sra
- ftree-switch-conversion
- ftree-tail-merge
- ftree-vrp

2) 对应 swcc 编译器-O2 选项。

d) -O3

描述:

在-O2 的基础上对代码运行速度进行进一步的优化。

备注：

1) 在-O2 基础上，-O3 使能了以下编译选项：

- fgcse-after-reload
- finline-functions
- fipa-cp-clone
- fpeel-loops
- fpredictive-commoning
- fsplit-loops
- fsplit-paths
- fstrict-aliasing
- ftree-loop-distribute-patterns
- ftree-loop-vectorize
- ftree-partial-pre
- ftree-slp-vectorize
- funswitch-loops

2) 对应 swcc 编译器-O3 选项。

3) 在-O3 选项的基础上打开-msimd 选项编译器会进行自动向量化的编译优化。

e) -Os

描述：

在-O2 的基础上对代码体积进行进一步优化。

备注：

1) 在-O2 基础上，-Os 使能了以下编译选项：

- finline-functions
- foptimize-strlen
- fschedule-insns

3) 对应 swcc 编译器-Os 选项。

2. -Ofast

描述：

该选项指示编译器不会严格遵循语言标准，相比-O3 选项，优化手段更为激进。

在-O3 基础上，-Ofast 使能了以下编译选项：

- fassociative-math
- fcx-limited-range
- ffinite-math-only
- freciprocal-math

在-O3 基础上，-Ofast 关闭了以下编译选项：

- fmath-errno
- ftrapping-math
- fsigned-zeros
- funsafe-math-optimizations

备注：

对应 swcc 编译器-Ofast 选项。

3. -ffast-math

描述:

该选项指示编译器不必严格遵守 IEEE754 标准, 允许编译器对浮点算数运算进行一定形式的变形, 以提高程序运行速度。对于精度要求较高的程序, 该优化可能产生结果错误。

备注:

- 1) 该选项仅在-Ofast 优化等级下打开;
- 2) 该选项打开以下优化选项:

-fno-math-errno

-funsafe-math-optimizations

-ffinite-math-only

-fno-rounding-math

-fno-signaling-nans

-fcx-limited-range

4. -ffp-contract=*style*

描述:

指示编译器是否进行浮点表示缩写, 例如浮点乘加指令优化等。

备注:

- 1) *style* 可以是 on/off/fast;
- 2) on 目前不支持, 效果同 off;
- 3) fast 指示编译器开启乘加指令优化, 默认在-O2 选项以上开启;
- 4) 对应 swcc 编译器-TARG:madd=on/off 选项。

5. -funroll-loops --param max-unroll-times=*n*

描述:

指示编译器进行循环展开, max-unroll-times 参数指示最大的循环展开次数。

备注:

- 1) 参数 max-unrolled-insns 定义了循环体中最大可以被展开的指令数; 参数 max-average-unrolled-insns 定义了循环体中最大可以被展开的指令数的平均值。对规模较大的循环体, 或者需要展开很多次的循环体, 适当增加这两个参数值, 有助于循环展开优化的使能。
- 2) 循环参数的类型设为大于 64 位的类型 (如 long、unsigned long、long long 和 unsigned long long), 有助于循环展开优化的使能。
- 3) 对应 swcc 编译器-LNO:outer_unroll_max,ou_max=*n* 选项。

6. -funroll-all-loops

描述:

指示编译器对循环体进行展开优化。

备注:

-funroll-all-loops 在不保证性能提升的前提下, 强行对循环体进行展开优化。

7. -fprefetch-loop-arrays

描述:

指示编译器对循环中的数组进行预取,将主存中数据预取到从核 Cache 中。

- 1) 预取优化默认打开循环展开优化, 可以通过调整循环展开次数影响预取效果。
- 2) 预取参数可以通过以下编译参数进行调整:

--params -prefetch-latency-slave=n 设定预取延迟为 n 条指令, 默认为 800。

该参数影响预取距离。

--params -simultaneous-prefetches-slave=n 同时预取的指令数为 n, 默认为 15。

--params -l1-cache-slave-size=n 从核 L1cache 大小, 单位为 KB, 默认为 32。

备注:

- 1) 对应 swcc 编译器-LNO:prefetch=n 选项;
- 2) 只支持从核代码。

8. -flto

描述:

指示编译器打开链接时优化。

备注:

使用链接时优化, 需要在编译和链接阶段都添加-flto 选项。

9. -mslave-lto

描述:

编译器支持从核代码链接时优化, 需要在编译从核文件及 hybrid 链接时使用。

示例:

```
$ swgcc -mhost -c host.c
```

```
$ swgcc -mslave -mslave-lto -c slave1.c slave2.c
```

```
$ swgcc -mhybrid -mslave-lto host.o slave1.o slave2.o
```

10. -fwhole-program

描述:

指示编译器将当前编译单元中的变量设置为外部不可见属性, 方便编译器进行更为激进的优化。

备注:

使用该选项可能导致对当前编译单元中变量的外部引用报未定义错误。

11. -fipa-type-escape -fipa-struct-peeling

描述:

指示编译器寻找程序中的结构体定义、声明及引用信息, 将访问次数较多的热点结构体域剥离, 组成新的结构体, 增强数据访问的局部性。

备注:

- 1) “访问次数较多”的阈值可配置;
- 2) --param struct-reorg-cold-struct-ratio=<value>, <value>取值介于 0-100, 默认为 10。

示例:

```
$ swgcc -fipa-type-escape -fipa-struct-peeling example.c
$ swgcc -fipa-type-escape -fipa-struct-peeling --param struct-reorg-cold-struct-ratio=20
example.c
```

12. -Wl,--gc-sections

描述:

编译时加上 -ffunction-sections、-fdata-sections 选项，链接时使用 -Wl,--gc-sections 选项，编译器会在链接时，取消对未使用到的数据段/代码段的链接。

备注:

目前已设置为编译器的默认选项。

13. -fbranch-predict-hint

描述:

面向指令预测 hint 指令的编译支持。

备注:

该选项仅对从核有效。由于硬件对于 hint 指令使用比较苛刻，在一些情况下，该选项不会提高转移预测性能；在最坏情况下，优化后也不会对降低程序原始性能。

14. -mpws

描述:

基于转移预测器方向切换的分支预测优化。

备注:

该选项仅对从核有效。此外该选项对程序分支特征要求比较严格，使用不当甚至可能严重降低程序性能。

15. -fmodule-ldm

描述:

module 名以 “ldm__” 开头的 module 中的变量都放置在 LDM 空间。

备注:

该选项为 swgfortran 选项。

示例:

```
module ldm__modx
integer aa
type node_struct
    real x,y,z
end type
type(node_struct) sss
end
```

加 -fmodule-ldm 选项编译生成的 mod，里面的 aa 和 sss 结构体都放置在从核 LDM 空间。

16. -mfma

描述:

编译器支持对乘加操作进行优化，利用硬件乘加单元，将乘法和加法两条指令合

并为一条乘加指令。

示例：

```
$ swgcc -mfma example.c
```

17. -msimd -ftree-vectorize

描述：

编译器支持主从核的自动向量化功能，使用方式如下：

编译时在-O2 选项优化的基础上加上-msimd -ftree-vectorize 选项或者直接使用-O3 -msimd 选项即可打开编译器的自动向量化功能。

示例：

- 1) \$ swgcc -msimd -ftree-vectorize example.c
- 2) \$ swgcc -O3 -msimd example.c

18. -fcache-hint

描述：

编译器支持从核 Cache 的写暗示优化，对连续大块数据的 Cache 写操作进行优化，提高 Cache 访问性能。

示例：

```
$ swgcc -mslave -fcache-hint slave1.c slave2.c
```

19. -fdynamic-uncache-symbols

描述：

编译器支持从核变量的可 Cache 属性和不可 Cache 属性的动态配置，通过编译选项加编译指导语句配合使用。

编译指导语句：区分数据默认是 cache 和 uncache 的两种情况，设置两组编译指导语句。

1、数据默认为 cached

#pragma uncache “xxx” ----将符号 xxx 访问变换为 uncache 访问。

#pragma resume_cache “xxx” ----将符号 xxx 访问恢复为 cache 访问。

2、数据默认为 uncached

#pragma cache “xxx” ---将符号 xxx 访问变换为 cache 访问。

#pragma resume_uncache “xxx” ---将符号 xxx 访问恢复为 uncache 访问。

3、编译指导语句参数说明

#pragma uncache “aa” 将符号 aa 的访问替换为 uncache 访问。

#pragma uncache “aa@@foo” 将函数 foo 内符号 aa 的访问替换为 uncache 访问。

备注：

备注：

- 1) 该机制只面向对处理器体系结构较为熟悉的高级用户；
- 2) 该机制只面向从核 Cache 结构；
- 3) 该机制不保证数据访问的正确性，需要在属性变化前后添加 Cache 刷新语句或者 Cache 淘汰语句来保证 Cache 与主存之间的数据一致性；
- 4) 该机制采用基于 pragma 实现的文本分析策略，一组编译指导语句确定了变换的空间，在此空间内的符号访问被转换为 uncache/uncached 访问，在此空间外，不做任何变换；

- 5) ”#pragma resume_xxx“为可选语句，如果不添加，则表示对应的”#pragma xxx“作用范围将持续到文本结尾；若为”#pragma xxx@@yyy“，则作用范围将持续到函数 yyy 文本结尾。（此处 xxx 表示 cache/uncache）。

示例：

```
$ swgcc -mslave -fdynamic-uncache-symbols slave.c
```

```
1 #include <stdio.h>
2 #include <slave.h>
3
4 int aa=3;
5 __uncached int dd=4;
6 __uncached int bb=5;
7 __uncached int cc[20];
8
9 int foo(){
10     #pragma cache lld@@foo"
11     flush_slave_cache();
12     int ret = aa+dd;
13     #pragma resume_uncache lld@@foo"
14     flush_slave_cache();
15     return ret;
16 }
17
18 int main()
19 {
20     printf("dd is %d\taa is 0x%x\n", dd, &aa);
21     aa = dd+4;
22     #pragma uncached lla@@main"
23     flush_slave_cache();
24     printf("dd is %d\taa is 0x%x\n", dd, &aa);
25     aa = bb+5;
26     cc[10] += foo();
27     #pragma resume_cache lla@@main"
28     #pragma cache bb"
29     flush_slave_cache();
30     printf("bb is %d\tbb addr is 0x%x\n", bb, &bb);
31
32     #pragma cache &c"
33     flush_slave_cache();
34     for(int i=0; i<20; i++)
35         printf("%d\n", cc[i]);
36 }
```

20. -lm、-lm_slave

描述:

编译器提供面向处理器体系结构特征优化的高性能快速数学库, 链接时使用-lm、-lm_slave 默认使用高性能数学库。

备注:

- 1) -lm 针对主核。
- 2) -lm_slave 针对从核。

示例:

- 1) \$ swgcc -mhost host1.o host2.o -lm
- 2) \$ swgcc -mslave slave1.o slave2.o -lm_slave
- 3) \$ swgcc -mhybrid host.o slave.o -lm -lm_slave

第四章 调试选项

1. -fverbose-asm

描述:

指示编译器在汇编文件中输出更多的调试信息, 包括:

- 1) 编译器版本及命令行选项;
- 2) 汇编指令对应的源代码行号;
- 3) 汇编指令对应的高级语言表达式信息, 例如内联函数信息等。

备注:

该选项仅适用于汇编文件生成过程。

2. -dp/-dP

描述:

指示编译器在汇编文件中输出指令模板信息。-dp 输出简要信息, -dP 输出完整信息。

备注:

该选项仅适用于汇编文件生成过程。

3. -g

描述:

指示编译器在生成的目标码中加入 DWARF 信息, 后续可以使用 gdb 工具进行调试。

备注:

使用该选项进行 gdb 调试, 优化级别建议设置为-O0。

4. -pg

描述:

指示编译器对主核程序进行插桩, 在运行时生成 profile 文件, 后续可以使用 gprof 工具进行热点函数分析。

备注:

用户也可以不加编译选项, 在运行时使用加载器的相关选项实现热点函数分析和重排优化, 并可同时针对主从核进行。

5. -fswperf

描述:

指示编译器对性能计数器进行插桩。

备注:

在链接时也需要使用该选项以添加相应库文件的支持。

6. -fdump-tree-all

描述:

指示编译器输出所有 gimple 优化遍信息。

7. **-fdump-rtl-all**

描述:

指示编译器输出所有 rtl 优化遍信息。

8. **-fdump-ipa-all**

描述:

指示编译器输出所有 ipa 优化遍信息。

第五章 其他选项

1. 为特定函数设置编译优化选项

a) `#pragma GCC optimize ("string", ...)`

描述:

该编译指导语句指示编译器为后续定义的函数设置新的编译优化选项, 新的编译选项由 “string” 指定, 不同的选项通过 ‘,’ 分隔。

备注:

- 1) 新的编译选项包括编译优化等级 (-O[0/1/2/3/s]及-Ofast, 需去除开头的 ‘-’) 以及与目标机器无关的编译优化选项 (以 “-f” 开头, 需去除开头的 “-f”), 与平台相关的选项 (以 “-m” 开头) 无法通过该指导语句进行设置;
- 2) 该指导语句设置的编译优化选项将应用于后续定义的所有函数。

示例:

```
#pragma GCC optimize("O2","cache-hint")
//后续函数的编译选项设置为-O2 -fcache-hint
void fun()
{
    .....
}
```

b) `#pragma GCC reset_options`

描述:

该编译指导语句指示编译器清除最近通过 `#pragma GCC optimize` 语句设置的编译选项, 将编译优化选项恢复为由命令行指定的缺省选项。

示例:

```
#pragma GCC optimize("O2","cache-hint")
//后续函数的编译选项设置为-O2 -fcache-hint
void fun_0()
{
    .....
}
#pragma GCC reset_options
//后续函数的编译选项恢复为通过命令行设置的缺省选项
void fun_1()
{
    .....
}
```

c) `__attribute__((optimize("string", ...)))`

描述:

该属性通知编译器为本函数设置新的编译选项, 新的编译选项由 “string” 指定, 不同的选项通过 ‘,’ 分隔。

备注:

- 1) 新的编译选项包括编译优化等级 (-O[0/1/2/3/s]及-Ofast, 需去除开头的 ‘-’) 以及与目标机器无关的编译优化选项 (以 “-f” 开头, 需去除开头的 “-f”), 与

平台相关的选项（以“-m”开头）无法通过该指导语句进行设置；

- 2) 该属性的作用范围仅为所处的函数定义，其余函数仍采用命令行设置的缺省选项。

示例：

```
__attribute__((optimize("O2","cache-hint")))  
//本函数的编译选项设置为-O2 -fcache-hint  
void fun_0()  
{  
    .....  
}  
//后续函数的编译选项恢复为通过命令行设置的缺省选项  
void fun_1()  
{  
    .....  
}
```

2. 自动向量化相关选项

a) -faggressive-ifcvt-for-vect

描述：

执行自动向量化的循环中含有条件语句时，添加此选项可以执行激进的条件语句转换从而实现自动向量化。

示例：

```
double sq[16];  
for(i=0;i<16;i++) {  
    sq[i] = (sq[i]>0.0)?sq[i]:0.0;  
    sq[i] = 1.0-sq[i];  
}
```

b) -funroll-innermost-loop

描述：

当有多层循环嵌套时，便一起在 O3 默认情况下会自动展开最内层循环，最内层循环展开后整个循环体中的语句数目变多，导致再以后的 pass 处理中难以正确分析出可以自动向量化的循环，从而导致不能自动向量化，添加该选项可以解决此问题。

示例：

```
real*8 cosths(30),sinths(30)  
do i=1,m  
    do j=1,n  
        cosths(j) = i + 0.01012  
        sinths(j) = i + 0.01015  
    enddo  
enddo
```

c) **-mtrunc**

描述:

当自动向量化的过程中需要将 float 类型转换为 double 类型时使用。

示例:

```
float a[16];  
double b[16];  
for (i = 0; i < 16; i++)  
    b[i] = a[i];
```

3. 功耗相关优化

a) **-freg-bypass**

描述:

面向寄存器旁路的指令调度优化。只支持从核代码，在-O2、-O3 情况下使能。

b) **-mslave-l0cache**

描述:

基于 L0 指令 cache 的指令调度优化。只支持从核代码，在循环展开优化遍（例如-funroll-loops）使能的情况下有效。

c) **-flow-power**

描述:

面向控制总线翻转的指令调度优化。只支持从核代码，在指令调度优化遍（例如-fschedule-insns、-fschedule-insns2）使能的情况下有效。